

BigQuery Feature Store

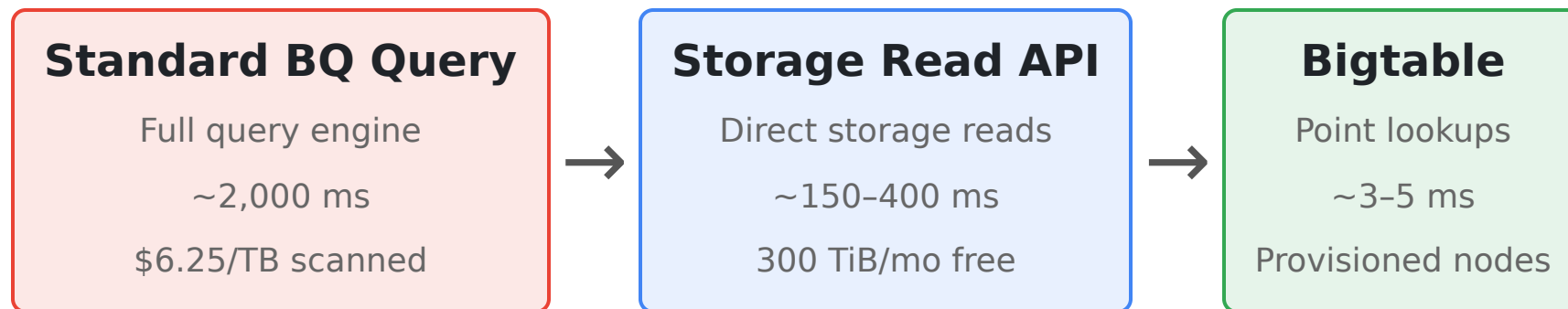
3 notebooks — no online store, no sync, no infrastructure

BigQuery Storage Read API | 20–200ms serving | Apache Arrow zero-copy

[github.com/statmike/vertex-ai-mlops/MLOps/Feature Store/BigQuery](https://github.com/statmike/vertex-ai-mlops/MLOps/Feature%20Store/BigQuery)

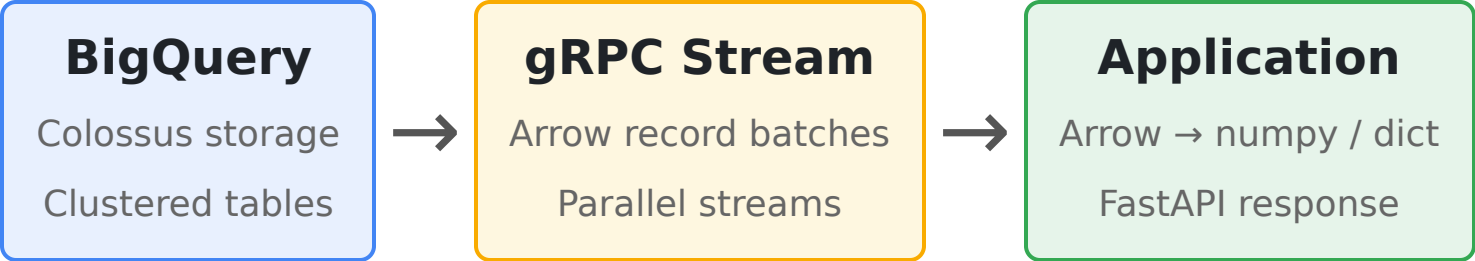
The Latency Sweet Spot

Standard BigQuery queries go through the full query engine — job scheduling, slot allocation, table scanning. The [Storage Read API](#) bypasses all of it, reading directly from BigQuery's Colossus storage layer via gRPC.



This approach lives in the sweet spot — fast enough for many serving workloads, flexible enough to replace complex pipelines, and cheap enough to run continuously.

Architecture



No online store, no sync pipeline, no provisioned infrastructure. BigQuery is both the offline store and the serving layer.

	Standard BQ Query	Storage Read API
Parse	SQL parse & plan	(skipped)
Allocate	Slot allocation	(skipped)
Read	Distributed scan	Direct storage read
Transfer	HTTP + deserialization	gRPC + Arrow zero-copy
Latency	~2,000 ms	~150-400 ms

The Data

A **130K-entity dataset** used across all three notebooks — 26 entity groups (A–Z), 5,000 entities per group, 224 columns spanning every BigQuery data type.

Table	Rows	Cols	Purpose
dense_features	130,000	224	Every BQ data type (FLOAT, STRING, STRUCT, JSON, ...)
sparse_events	2,600,000	10	Time-series events (20 per entity)
dense_features_clustered	130,000	224	Primary table — clustered by group + ID
entity_embeddings	130,000	3	64-dim vectors for similarity search

Why clustering matters: `CLUSTER BY entity_group, entity_id` physically sorts data so `row_restriction` filters prune at the storage block level.

Core Pattern — Row Restriction + Column Pruning

Row restriction — any WHERE clause

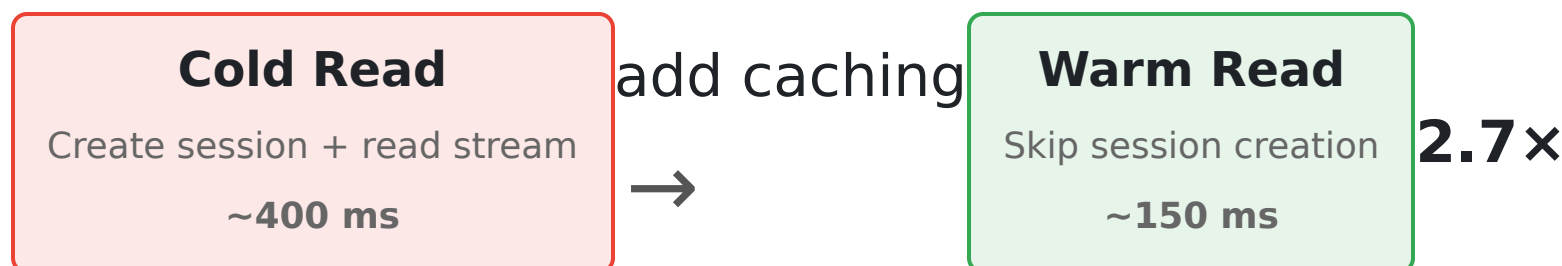
Filter	Rows	Latency
<code>group = 'A' AND id = '000001'</code>	1	~350 ms
<code>feature_float_1 > 50.0</code>	2,477	~400 ms
<code>group IN ('A','B','C') AND float_1 > 50</code>	3,794	~406 ms
<code>id BETWEEN '002000' AND '003000'</code>	1,000	~396 ms

Column pruning — read only what you need

Columns	Latency	Data
5	~630 ms	270 KB
50	~524 ms	2 MB
224 (all)	~1,242 ms	10 MB

Session Caching

Session creation is the dominant latency cost — ~60% of total read time.
The `FeatureSessionPool` eliminates it for repeat reads.

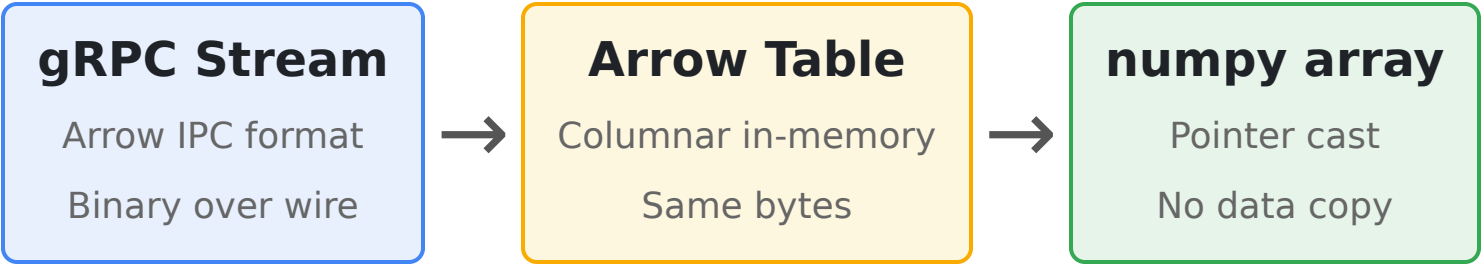


`FeatureSessionPool` — 48 lines of Python, no external dependencies.

- TTL-based eviction (default 5 minutes)
- `warm()` method for pre-creating sessions at startup
- Reuses the gRPC channel and connection pool across requests
- Copy into any project — just a Python dict with TTL

Arrow Zero-Copy

Arrow's on-wire format matches its in-memory format — the bytes that arrive over gRPC are the bytes your code works with.



Conversion benchmarks (5,000 rows):

Target	Time	Notes
Arrow → numpy	235 µs	Zero-copy for float columns
Arrow → pandas	3,065 µs	Memory copy
Arrow → Python dict	20,384 µs	Python object allocation

Serving Application

FastAPI + `asyncio.to_thread()` + `FeatureSessionPool` — non-blocking reads with warm gRPC channels.

Endpoints

- `GET /features/{group}/{id}` — single entity
- `POST /features/batch` — N sessions via `asyncio.gather`
- `POST /features/batch/combined` — 1 session, `IN (...)` clause

Performance (10 entities)

Pattern	Total	Per-entity
Sequential (10× GET)	1,538 ms	154 ms
Concurrent (gather)	448 ms	45 ms
Combined batch	396 ms	40 ms

Concurrent (20 in-flight): p50 359 ms, p95 587 ms

Batch Patterns — Per-Entity vs Combined

Per-entity batch

N ReadSessions via `asyncio.gather()`

- 1 roundtrip, N server-side sessions
- Latency scales with entity count

Combined-restriction batch

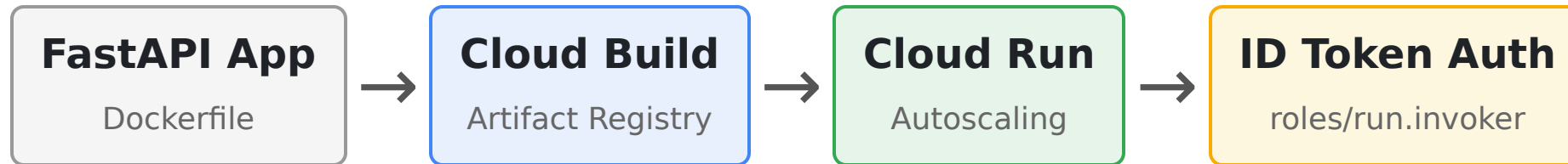
1 ReadSession with `IN (...)` clause

- 1 roundtrip, 1 server-side session
- Near-constant latency

Combined wins at >10 entities.

Entities	Per-Entity	Combined	Speedup
1	~453 ms	~453 ms	1.0×
10	~800 ms	~520 ms	1.5×
25	~1,400 ms	~540 ms	2.6×
50	~1,900 ms	~510 ms	3.7×
100	~2,289 ms	~486 ms	4.7×

Cloud Run Deployment



Performance

Metric	Value
Cold start	~229 ms
Warm p50	~453 ms
Throughput plateau	~30-40 req/s
Spike bursts (200 sim.)	Zero errors

Production configuration

Setting	Value
<code>min_instance_count</code>	1
<code>max_instance_count</code>	10
<code>max_instance_request_concurrency</code>	20
CPU / Memory	2 vCPU / 2 GiB

Traffic splitting for canary deployments — revision-based rollout of new feature columns.

Three-Way Comparison

Operation	BQ Query	Storage API	Bigtable (ref)
1 entity (5 features)	~2,000 ms	~350 ms	~3-5 ms
100 entities	~1,500 ms	~460 ms	~50-100 ms
1,000 entities	~1,600 ms	~420 ms	~200-500 ms
5,000 entities	~1,500 ms	~360 ms	~500-2,000 ms
Filtered segment	~1,600 ms	~410 ms	N/A

Component	BQ Storage API	Bigtable
Infrastructure	\$0 (serverless)	~\$500+/mo (1 node min)
Read cost	300 TiB/mo free	Included in node cost
Sync pipeline	None	EXPORT DATA / Dataflow

When to Use Which

Sub-ms atomic writes (counters, real-time state)? → Bigtable

Single-digit-ms point lookups? → Bigtable

20-200ms OK? Want simplicity + governance? → BQ Storage API

Managed feature registry? → Vertex AI Feature Store

Capability	BQ Storage API	Bigtable	Vertex AI FS
Point-lookup latency	20-200 ms	<10 ms	Managed
Bulk reads (1K+)	Excellent	Many RPCs	Managed
Filtering	Predicate pushdown	Row key only	Feature Views
Streaming writes	Storage Write API	Atomic point writes	Offline-first
Governance	Built-in (Dataplex)	You build it	Managed
Infrastructure	None	Provisioned	Managed

Resources

Repository: github.com/statmike/vertex-ai-mlops/MLOps/FeatureStore/BigQuery

Storage Read API: cloud.google.com/bigquery/docs/reference/storage

Apache Arrow: arrow.apache.org

Clustered tables: cloud.google.com/bigquery/docs/clustered-tables

Cloud Run: cloud.google.com/run/docs/overview/what-is-cloud-run